

CSE450 Capstone Project Report

Title of the Project

Submitted by

Student Name	Student ID
Student Name	Student ID
Student Name	Student ID
Student Name	Student ID
Student Name	Student ID

Supervised by

Supervisor Name
Designation, Department of CSE, BUET



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Session 20XX–20XX
Month YYYY

CANDIDATES' DECLARATION

We declare that the work presented in this report, titled "Title of the Project", is the outcome of the engineering work carried out by us under the supervision named on the title page.

All prior work, third-party code, libraries, data, copyrighted material, and tools used in the project have been properly acknowledged. Any use of generative AI tools has been disclosed. We accept accountability and responsibility for the accuracy and integrity of this report and for our individual contributions to the project.

Student Name (ID)

Student Name (ID)

Student Name (ID)

Student Name (ID)

Student Name (ID)

ACKNOWLEDGEMENT

Thank your supervisor, and anyone else who helped: domain experts, collaborators, users who tested the system, people from other departments or organisations you consulted.

ABSTRACT

In no more than one page, state the problem, what was designed and built, how it was verified or validated, and the key outcomes. For a research-oriented project, briefly include the method and principal findings.

Keywords: *three to six keywords*

Contents

Candidates' Declaration	i
Acknowledgement	ii
Abstract	iii
1 Problem Identification and Formulation	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Complexity of the Problem	2
1.4 Project Objectives and Scope	4
2 Preliminary Research	6
2.1 Literature Review	6
2.2 Existing Solutions and Technologies	6
2.3 Market and User Research	7
2.4 Feasibility Study	7
3 Requirements Analysis	9
3.1 Project Objectives and Constraints	9
3.1.1 Problem Context and Origin	9
3.1.2 Requirements Elicitation	9
3.1.3 Stakeholders and Their Expectations	10
3.1.4 Conflicting Expectations and Their Resolution	10
3.1.5 Project Objectives	12
3.1.6 Functional Requirements	13

3.1.7	Non-Functional Requirements	17
3.1.8	Constraints	19
3.2	Regulatory Requirements and Standards	21
3.3	Formulation of Solutions	21
4	System Design and Implementation	22
4.1	Functional Design and Sub-problem Partitioning	22
4.2	Design Refinement and Simulation	22
4.3	Application of Tools	22
4.4	Implementation	22
4.5	Deployment	23
5	Investigation and Evaluation	24
5.1	Experimental Investigation	24
5.2	Performance Evaluation	24
5.3	Deviations and Design Revisions	24
6	Project Management, Teamwork, and Impact	25
6.1	Project Management and Finance	25
6.1.1	Planning and Risk Management	25
6.1.2	Budgeting and Resource Identification	25
6.1.3	Economic Analysis and Financial Prospecting	25
6.1.4	Sustainability in Business and Commercialisation	26
6.2	Individual and Teamwork	26
6.2.1	Participation and Contribution	26
6.2.2	Collaboration and Conflict Resolution	26
6.2.3	Leadership and Direction	26
6.2.4	Multidisciplinary Engagement	26
6.3	Impact on Society	26
6.4	Environmental Impact and Life Cycle Sustainability	27

6.5	Ethics	27
6.5.1	Equity and Inclusivity	27
6.5.2	Accountability and Personal Responsibility	27
6.5.3	Proper Use of Intellectual Property	27
6.5.4	Professionalism and Ethical Codes	27
7	Conclusion	28
7.1	Conclusion and Future Work	28
7.2	Continuous Engagement and Self-directed Learning	28

List of Figures

4.1	System architecture	22
6.1	Project schedule	25

List of Tables

1.1	Complex engineering problem characteristics (P1–P7) and evidence from this project	2
1.2	Knowledge profiles referenced by P1	4
3.1	Stakeholders and their principal expectations	10
3.2	Conflicting stakeholder expectations and their resolution	11
3.3	Project objectives	12
3.4	Functional requirements: authentication, tenancy and access control	13
3.5	Functional requirements: workspace and project configuration	14
3.6	Functional requirements: metric ingestion	14
3.7	Functional requirements: health scoring and presentation	15
3.8	Functional requirements: team feedback surveys	16
3.9	Functional requirements: management action log	17
3.10	Non-functional requirements	17
3.11	Project constraints	19

Chapter 1

Problem Identification and Formulation

1.1 Background and Motivation

A software organisation runs each of its projects through several specialised tools: a version-control platform for source code and code review, an issue-tracking platform for planning and sprint management, a static code-analysis service for code quality, and a CI/CD service for builds and deployments. Each tool exposes its own dashboard, reports its own metrics in its own vocabulary, and answers questions only about its own slice of the work. None of them answers the question that engineering leadership actually asks: *is this project, taken as a whole, healthy, and is it improving or deteriorating?*

The consequence, as described by Spectrum Software & Consulting Pvt. Ltd. (SSCL), the industry partner that originated this project, is that assessing the state of a single project means opening four to five dashboards in turn and reconciling their readings by judgement. Doing this across a whole portfolio is impractical, so it is done infrequently. As a result, a project that is gradually deteriorating is often noticed only after the damage has become visible as missed deadlines or failing builds.

Two further gaps compound this. First, tool metrics describe what the code and the pipeline are doing, but not how the team is doing; low morale, persistent blockers, and loss of confidence in a plan are often the earliest signals of a project in trouble, and they appear in no dashboard. Second, when a manager does identify a problem and acts on it, the action is recorded nowhere, so there is no later way to tell whether the intervention worked.

Pulse addresses these gaps. It ingests metrics from the separate tools, normalises them into comparable health scores on a single scale, keeps every score explainable in terms of the metrics that produced it, collects anonymous team feedback that metrics cannot capture, and records the management actions taken in response so that their effectiveness can be reviewed. It is designed as a trend indicator, revealing how projects move over weeks and months, rather than as a real-time monitor.

1.2 Problem Statement

Engineering leadership and project managers at software organisations such as SSCL oversee many projects at once, and each project produces its data in several independent tools: version control, issue tracking, static code analysis and CI/CD. Every tool reports its own metrics on its own scale, and none of them offers a common notion of whether a project is healthy. To judge a project's condition, a manager has to gather readings from each tool separately and reconcile them by hand, and the tools capture nothing about how the team itself is faring or about what management did when a problem was found.

Because this reconciliation is slow and manual, it is done infrequently and inconsistently; projects cannot be compared on a like-for-like basis; and a project that is gradually deteriorating is usually recognised only once the damage has become visible. The early warning signs that only the team is aware of, such as people losing motivation or work that keeps getting stuck, never reach any dashboard, and because interventions are not recorded, whether they helped is never known. What is missing is a single, comparable and explainable view of project health that draws on all of these tools, that captures the team's own signal without exposing individuals, and that connects management action to measured outcome. Closing that gap is the problem this project addresses.

1.3 Complexity of the Problem

Table 1.1: Complex engineering problem characteristics (P1–P7) and evidence from this project

Code	Definition	Evidence from this project
P1	Depth of knowledge required. Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6, or K8, which allows a fundamentals-based, first-principles analytical approach.	K3: the weighted-normalisation scoring model, its handling of missing inputs, and its exact decomposition into per-metric contributions. K4 and K6: asynchronous job processing with retries and idempotent scheduling; rate-limit-aware integration with third-party REST and GraphQL APIs; credential encryption, password hashing and session security. K5: the connector abstraction that isolates provider-specific ingestion from scoring, and the snapshot-based data model that makes every score reproducible.

Code	Definition	Evidence from this project
P2	Range of conflicting requirements. Involves wide-ranging or conflicting technical, engineering, and other issues.	<i>To be completed after discussion with the team</i>
P3	Depth of analysis required. Has no obvious solution and requires abstract thinking and originality in analysis to formulate suitable models.	There is no established mapping from heterogeneous tool metrics to a single comparable scale. The team had to derive, per metric, a normalisation function and weight, define how absent metrics are treated, and design the scores so that they remain exactly explainable; and had to design survey aggregation that yields team-level insight without any per-respondent record.
P4	Familiarity of issues. Involves infrequently encountered issues.	<i>To be completed after discussion with the team</i>
P6	Extent of stakeholder involvement and conflicting requirements. Involves diverse groups of stakeholders with widely varying needs.	Seven stakeholder groups (Table 3.1): the client's leadership, its managers, its developers as data subjects, the tenant administrator, the academic supervisor, external service providers as constraint setters, and the development team, whose needs conflict as set out under P2.
P7	Interdependence. Is a high-level problem including many component parts or sub-problems.	Authentication and tenancy; workspace and connector configuration; metric ingestion and scheduling; scoring; survey generation, distribution and analysis; and the action log. These are dependent: scoring requires ingestion, survey generation is driven by scores and their trends, actions are recorded against both, and scheduling must space projects that share a credential to stay within rate limits.

Table 1.2: Knowledge profiles referenced by P1

Code	Description
K3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
K4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
K5	Knowledge that supports engineering design in a practice area.
K6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
K8	Engagement with selected knowledge in the research literature of the discipline.

1.4 Project Objectives and Scope

The project set out to achieve the following objectives:

1. **Consolidate** the metrics of a project's version-control, issue-tracking, code-analysis and CI/CD tools into a single dashboard, so that routine status assessment needs none of the tools' own dashboards.
2. **Normalise** those heterogeneous metrics into seven health scores on a common 0–100 scale, plus an overall score, so that any two projects can be compared directly.
3. **Explain** every score on request: which metrics produced it, their measured values, their weights, and each one's contribution.
4. **Capture** qualitative team feedback through anonymous surveys, with no stored link between a respondent and their answers.
5. **Record** the management actions taken in response to identified problems, and prompt for a review of their effectiveness after a set interval.
6. **Accumulate** health history automatically on a fixed schedule, so that trends build up without anyone triggering a synchronisation.
7. **Remain extensible**, so that a further tool provider can be added without changing the scoring logic.

Scope. The following is within the scope of the project:

- One connector for each tool category: GitHub (version control), Jira (issue tracking), SonarCloud (code quality) and GitHub Actions (CI/CD).
- Multi-tenant organisations with administrator and member roles; workspaces bound to a version-control organisation and access token; import of repositories as projects.
- On-demand and scheduled synchronisation, an immutable snapshot history, and portfolio and per-project dashboards with trends and score breakdowns.
- Automatically generated, editable survey questions; one scheduled survey per project per month plus on-demand surveys; anonymous responses; and survey analysis.
- A management action log with keyword and semantic search and effectiveness review.
- Email delivery of verification codes, password resets, invitations and surveys, and broadcast of survey links to Slack and Discord.

The following is explicitly outside the scope:

- Real-time monitoring; the system is a trend indicator operating on a scheduled cadence.
- Version-control providers other than GitHub and CI/CD systems other than GitHub Actions; the connector abstraction allows them, but they were not implemented.
- Any per-individual performance measurement, ranking or surveillance of developers.

Chapter 2

Preliminary Research

2.1 Literature Review

A formal literature review is not applicable to this project, which is an applied engineering effort rather than a research contribution.

2.2 Existing Solutions and Technologies

SSCL, like most software organisations, already relies on a set of specialised tools that each cover one part of this problem: GitHub for version control, Jira for issue tracking, SonarQube for static code analysis, and GitHub Actions for CI/CD. Each is capable and widely adopted, but each is scoped to its own concern and reports through its own dashboard; none aggregates across the others, which is exactly the gap described in Chapter 1.

A category of commercial “engineering intelligence” platforms – LinearB, Swarmia, Faros AI, and Jellyfish among them – does aggregate metrics across the delivery pipeline, mostly throughput and cycle-time figures pulled from version control and issue tracking. They integrate with the same tools SSCL already uses and produce cross-tool dashboards. However, they focus almost entirely on delivery velocity and code metrics: none incorporates the team’s own qualitative signal through anonymous surveys, none records management actions taken in response to a decline or tracks whether they worked, and all are priced per seat. Static-analysis and CI dashboards (SonarCloud, GitHub Actions) remain confined to their own tool with no cross-tool scoring at all.

The gap is a single, explainable health score spanning all four tool categories, combined with anonymous team feedback and a record of management action and its effectiveness.

2.3 Market and User Research

The primary users are engineering leaders and project managers at software organisations that run multiple concurrent projects across a mix of specialised tools – version control, issue tracking, static analysis, and CI/CD – for each one. SSCL, the project’s industry partner, is representative of this user base and was the direct source of evidence for the need: its engineering leadership described project-health assessment as a routine but repetitive task of opening several dashboards on a fixed cadence and reconciling them by judgement, with no tool available to aggregate them and no channel through which a team’s own sense of how a project was going ever reached management.

Because SSCL was the only source of user evidence available to the team, no formal market sizing was attempted. The underlying situation, however, is not specific to SSCL: any organisation running comparable multi-tool, multi-project engineering operations faces the same fragmentation, which suggests demand extends beyond the immediate industry partner.

2.4 Feasibility Study

Technical. GitHub, Jira, SonarCloud and GitHub Actions each expose documented REST or GraphQL APIs with free-tier rate limits sufficient for a handful of pilot projects, so all the data the scoring model needs was obtainable without purchasing access. The team’s existing experience with Node.js, TypeScript, Supabase Postgres and Redis/BullMQ meant the pipeline could be built on a stack already familiar to it. The main technical risk, rate limits shared across projects on the same credential, was manageable by spacing scheduled syncs. Free hosting tiers covered the API and frontend; the one exception is the background worker, which must run continuously rather than on demand and currently has no equivalent free serverless option, so it remains unhosted.

Economic. The system was built entirely on free tiers – Supabase, Vercel, Upstash Redis, and the free quotas of the integrated tools – so development introduced no cost beyond the team’s time. For SSCL, the expected benefit is reducing the recurring manual effort of dashboard reconciliation across its portfolio, which comes at no licensing cost to adopt.

Operational. Adoption only requires the access tokens SSCL already holds for GitHub, Jira and SonarCloud and a short one-time setup per project; the underlying tools continue to be used exactly as before, so operational disruption is minimal.

Legal and regulatory. Access tokens are stored encrypted, and survey responses are collected with no stored link to a respondent, so no personally identifiable data is introduced beyond ordinary account information.

The project was judged feasible on the assumption that SSCL continues to provide the access

tokens and project structure it already uses; the completed build, delivered within the team's two-semester timeline, confirms that assessment.

Chapter 3

Requirements Analysis

3.1 Project Objectives and Constraints

3.1.1 Problem Context and Origin

The project originated with Spectrum Software & Consulting Pvt. Ltd. (SSCL), a software development organisation. Engineering leadership at SSCL monitored the health of their software projects through four to five separate tool dashboards such as a version control platform (e.g., Github), an issue-tracking and sprint-planning platform (e.g., Jira), a static code-analysis service (e.g., SonarQube), and a CI/CD service (e.g., Jenkins). Each dashboard presented its own metrics in its own vocabulary, with no shared notion of whether a project was, overall, in good or poor health. Assessing the state of a single project required visiting every dashboard in turn and mentally reconciling the results; assessing a portfolio of projects this way was impractical.

SSCL proposed a platform that would ingest data from these separate tools, translate heterogeneous raw metrics into a common, comparable scale, and present the result in a single view. The system described in this report addresses that problem. It is explicitly positioned as a trend indicator system rather than a real-time monitoring tool. Its purpose is to reveal how projects and teams are trending over weeks and months, not to report the state of a build in the last five minutes.

3.1.2 Requirements Elicitation

Requirements were gathered through three complementary channels. The primary channel was a series of meetings with the Chief Technical Officer of SSCL, who originated the concept and articulated the operational pain point, the tools in use, and the decisions the system was intended to support. A secondary channel was an intern at SSCL with prior exposure to the concept, who provided practical grounding on how the organisation's teams actually work day to day. The third channel was the team's own analysis such as study of the public APIs exposed by the candidate tools to establish which metrics were obtainable in practice, and derivation of a scoring model from that feasible metric set. Throughout, the university supervisor provided guidance on scope, methodology and academic rigour.

3.1.3 Stakeholders and Their Expectations

Table 3.1: Stakeholders and their principal expectations

ID	Stakeholder	Role	Principal expectations
SH-01	SSCL leadership	Client; originator; primary requirement source	A single consolidated view replacing four to five dashboards; comparable scores across projects; early visibility of declining projects
SH-02	Engineering managers and project managers	Primary end users	Status at a glance; the ability to drill into a score that looks wrong; low operational overhead
SH-03	Developers employed by SSCL	Data subjects and survey respondents	Assurance that the system is not an instrument of individual surveillance or ranking; minimal interruption; the ability to answer candidly without attribution
SH-04	Company administrator	Tenant and credential owner	Control over access tokens, workspace membership and tool configuration; assurance that credentials are not exposed
SH-05	University supervisor	Academic oversight	Methodological rigour, documented decision-making, demonstrable evaluation
SH-06	External tool and service providers	Constraint setters	Adherence to their authentication models, rate limits, free-tier quotas and terms of service
SH-07	Development team	Implementers	A scope deliverable by five students within two semesters

3.1.4 Conflicting Expectations and Their Resolution

Several stakeholder expectations are in genuine tension. Each conflict was resolved by an explicit requirement rather than left to implementation judgement.

Table 3.2: Conflicting stakeholder expectations and their resolution

#	Conflict	Resolution	Encoded by
1	SH-01 seeks insight into how teams are performing; SH-03, the same organisation's employees, require that the system not attribute opinions to individuals	Qualitative feedback is collected anonymously, with no stored link between a respondent and their answers, and aggregate insight is withheld below a minimum response count	NFR-07, NFR-08
2	SH-01 and SH-02 would benefit from frequent team feedback; SH-03 experience repeated surveys as interruption and survey fatigue	Survey frequency is capped per project per month, and a cooldown is enforced per developer across all projects, with fair rotation for developers on several projects	FR-33, FR-31
3	SH-01 expects current data; SH-06 impose rate limits that make continuous polling infeasible under free-tier quotas	The product is positioned as a trend indicator on a scheduled cadence rather than a real-time monitor, with on-demand sync available when a current reading is needed	FR-13, FR-18, NFR-16
4	SH-01 expects every project to carry a comparable score; in practice most projects have only a subset of the four tool categories configured	Scores are computed from whichever inputs are available, with the weight of absent inputs redistributed proportionally rather than treated as zero	FR-22
5	SH-04 must be able to verify and rotate access tokens; the same tokens must never be casually exposed	Credentials are masked by default in all responses and revealed only through an explicit, administrator-restricted action, and are encrypted at rest	FR-12, NFR-01, NFR-04
6	SH-01 expects the analytical capability that AI features provide; C-03 (in Section 3.1.8) permits no expenditure, and external AI services may be unavailable	AI-dependent features are optional, the system degrades to non-AI behaviour when such services are unconfigured or unavailable, rather than failing	NFR-13

#	Conflict	Resolution	Encoded by
7	SH-01 requested coverage of their full toolchain; SH-07 capacity permits only a subset within the available time	A representative connector from each of the four tool categories was implemented, behind an abstraction that permits further providers to be added without altering scoring logic	NFR-21
8	SH-05 values methodological rigour and documentation; SH-01 values a working, maintainable product	Both were treated as delivery criteria: design decisions are documented and the scoring rules are specified and unit-tested, which serves the academic requirement while improving maintainability for the client	NFR-22, NFR-23

3.1.5 Project Objectives

Table 3.3: Project objectives

ID	Objective
O-01	Consolidate engineering metrics from independent tools into a single dashboard, removing the need to consult each tool's own dashboard for routine status assessment
O-02	Normalise heterogeneous raw metrics into comparable, interpretable health scores on a uniform 0–100 scale across seven quality dimensions
O-03	Keep every score explainable and auditable, so that a manager can determine which underlying metrics produced a score and how much each contributed
O-04	Capture qualitative team signal via survey that quantitative metrics cannot express, without compromising the anonymity of individual respondents
O-05	Close the management loop by recording the actions taken in response to identified problems and reviewing their effectiveness after a defined interval

ID	Objective
O-06	Accumulate project health history automatically on a fixed schedule, so that trend data is evenly spaced and does not depend on users manually triggering synchronisation.
O-07	Remain extensible to additional tools and providers without modification of the core scoring logic

Objectives O-03 and O-05 distinguish the system from a conventional metrics dashboard, the first makes scores accountable rather than opaque, and the second records the managerial response alongside the measurement, enabling later assessment of whether interventions worked.

3.1.6 Functional Requirements

Authentication, Tenancy and Access Control

Table 3.4: Functional requirements: authentication, tenancy and access control

ID	Requirement
FR-01	The system shall allow a new organisation to register, creating a company and an initial administrator account, with the administrator's email address verified by a one-time code before the account is created
FR-02	The system shall authenticate users by email address and password and maintain an authenticated session
FR-03	The system shall allow a user to reset a forgotten password through a single-use emailed link, and shall invalidate all of that user's existing sessions upon reset
FR-04	The system shall allow an administrator to invite a person to a project by email address, using a single-use invitation that the recipient accepts either by registering or by accepting while already authenticated
FR-05	The system shall support two roles, administrator and member, where a member may access only the projects to which they are assigned and an administrator may access all projects within their company
FR-06	The system shall allow an administrator to remove a member from a project

Workspace and Project Configuration

Table 3.5: Functional requirements: workspace and project configuration

ID	Requirement
FR-07	The system shall allow an administrator to create a workspace defined by a version-control provider, an organisation identifier and an access token
FR-08	The system shall list the repositories reachable with a supplied access token before any data is stored, so the administrator can confirm the token's scope
FR-09	The system shall import selected repositories as projects belonging to the workspace
FR-10	The system shall allow further repositories to be imported into an existing workspace, excluding those already imported
FR-11	The system shall allow an administrator to configure per-project integrations for project management, code quality and CI/CD tools, with the version-control credential inherited from the workspace
FR-12	The system shall mask credential values in all responses by default, and shall reveal a credential only through an explicit administrator-restricted request

Metric Ingestion

Table 3.6: Functional requirements: metric ingestion

ID	Requirement
FR-13	The system shall allow a user to trigger an immediate synchronisation of a project across all of its configured tools
FR-14	The system shall retrieve metrics from a version-control platform, an issue-tracking platform, a static code-analysis service and a CI/CD service
FR-15	The system shall record each synchronisation as an immutable, timestamped project snapshot with the metrics retrieved from each tool
FR-16	The system shall report synchronisation progress to the user interface in real time, indicating the status of each tool individually
FR-17	The system shall allow a user to navigate away from or reload the page during a synchronisation and reattach to its progress

ID	Requirement
FR-18	The system shall automatically synchronise every configured project on a configurable daily schedule without user intervention
FR-19	The system shall complete a synchronisation and record the data successfully retrieved when one or more individual tools fail, reporting the outcome as partial

Health Scoring and Presentation

Table 3.7: Functional requirements: health scoring and presentation

ID	Requirement
FR-20	The system shall compute seven health scores per snapshot — security, reliability, maintainability, CI/CD and deployment health, team health, engineering process, and planning and execution — each on a 0–100 scale where a higher value indicates better health
FR-21	The system shall compute an overall health score for each snapshot from the seven dimension scores
FR-22	The system shall compute scores from the metrics actually available for a snapshot, redistributing the weight of unavailable metrics across those present
FR-23	The system shall persist computed scores against the snapshot from which they were derived
FR-24	The system shall present a portfolio view listing all accessible projects with their overall score, dimension scores and trend
FR-25	The system shall present a per-project dashboard showing the current score, its trend, the dimension scores and their history over time
FR-26	The system shall present, on request, a breakdown of any dimension score identifying the contributing metrics, their measured values, their applied weights and their individual contributions

Team Feedback Surveys

Table 3.8: Functional requirements: team feedback surveys

ID	Requirement
FR-27	The system shall generate survey questions automatically, informed by the project's current health scores, their trend since the previous synchronisation, and notable operational conditions detected in the latest metrics
FR-28	The system shall filter generated questions for duplication and assess their quality, discarding those below a configurable quality threshold
FR-29	The system shall allow an authorised reviewer to edit the generated questions before dispatch, and shall prevent further editing once the survey has been distributed
FR-30	The system shall distribute one scheduled pulse survey per project per month, at a time selected within a configurable window and fixed once assigned
FR-31	The system shall allow an authorised user to dispatch an additional survey on demand, subject to a configurable monthly limit per project
FR-32	The system shall distribute a survey as a single anonymous link, broadcast to configured team communication channels and emailed to eligible project developers
FR-33	The system shall not email a developer a survey within a configurable cooldown period of their previous survey email, and shall rotate fairly between the projects a developer belongs to
FR-34	The system shall accept anonymous responses through the time-limited link that expires at a configurable deadline
FR-35	The system shall close a survey at its deadline or on demand and produce an analysis comprising dimension scores, recurring themes, a narrative summary and a per-question summary
FR-36	The system shall flag a project as due for a survey when defined health scores fall below configured thresholds

Management Action Log

Table 3.9: Functional requirements: management action log

ID	Requirement
FR-37	The system shall allow a user to record a management action comprising the problem observed, its assessed cause, the action taken, the date, and the projects affected
FR-38	The system shall list recorded actions per project and across the company, scoped to the user's role
FR-39	The system shall allow actions to be searched by keyword, and shall additionally offer a semantic search that retrieves conceptually related actions
FR-40	The system shall prompt the author of an action to rate its effectiveness after a defined review interval, and shall allow that review to be deferred
FR-41	The system shall allow a member to modify or delete actions they recorded, and an administrator to do so for any action within their company

3.1.7 Non-Functional Requirements

Table 3.10: Non-functional requirements

ID	Requirement
<i>Security</i>	
NFR-01	Access tokens and credential material shall be encrypted at rest
NFR-02	User passwords shall be stored only as salted cryptographic hashes and shall never be recoverable
NFR-03	Session identifiers shall be transmitted in cookies inaccessible to client-side scripts, shall expire after a defined period of inactivity, and shall be revocable
NFR-04	Credential values shall not appear in API responses except through the explicit reveal action of FR-12, and shall never be written to application logs
NFR-05	Endpoints accessible without authentication shall be rate-limited
NFR-06	Survey access links shall be tamper-evident, such that modification of an embedded identifier or deadline invalidates the link

ID	Requirement
<i>Privacy</i>	
NFR-07	A survey response shall not be attributable to the individual who submitted it, and no stored association between respondent identity and response content shall exist
NFR-08	Aggregate survey insight shall be reported only where the number of responses meets a configured minimum
NFR-09	Data transmitted to third-party analysis services shall not include the names of individuals or identifiers of specific work items
<i>Reliability</i>	
NFR-10	The failure of one tool during a synchronisation shall not prevent the remaining tools from being processed and recorded
NFR-11	Background jobs shall be retried automatically on failure with an increasing delay between attempts
NFR-12	Scheduled synchronisation shall be idempotent, such that a repeated or retried schedule trigger does not duplicate work for the same project and time slot
NFR-13	The system shall degrade gracefully when optional external services are unconfigured or unavailable, substituting reduced functionality rather than failing
NFR-14	Multi-step operations that cannot be performed atomically shall reverse their partial effects on failure
<i>Performance and Efficiency</i>	
NFR-15	Calls to external APIs shall be issued with bounded concurrency
NFR-16	The system shall observe the rate limits published by external providers, and shall stagger projects that share a credential so that a scheduled run does not exhaust a single credential's quota
NFR-17	Dashboard views shall be served from stored snapshot data and shall not block on live calls to external tools
<i>Usability</i>	
NFR-18	A manager shall be able to assess routine project status without opening any of the underlying tools

ID	Requirement
NFR-19	The progress of a running synchronisation shall be visible to the user, with per-tool status
NFR-20	Any presented score shall be explainable on demand in terms of its contributing metrics
<i>Maintainability</i>	
NFR-21	Support for an additional tool provider shall be addable without modification of the scoring logic
NFR-22	The scoring rules for each health dimension shall be documented and covered by automated tests
NFR-23	Every change shall be validated automatically for type correctness, static analysis compliance, successful build and passing tests before it is merged
<i>Portability and Deployability</i>	
NFR-24	All environment-specific configuration shall be supplied externally, with no credential or endpoint hard-coded in source
NFR-25	The server components shall be deployable as containers
NFR-26	Scheduling shall be configurable to an arbitrary named time zone, independent of the host's clock

3.1.8 Constraints

Table 3.11: Project constraints

ID	Category	Constraint and implication
C-01	Time	Development spanned two academic semesters, approximately twelve months, undertaken alongside concurrent coursework rather than full-time. This required incremental delivery and forced explicit deferral of lower-priority capability
C-02	Human resources	The team comprised five students with no dedicated operations, quality-assurance or design specialists; all such responsibilities were absorbed by the same five members

ID	Category	Constraint and implication
C-03	Budget	No monetary budget was available. Every external service — database, cache, hosting, AI inference, email delivery and code analysis — had to operate within a free service tier. This constrained hosting to a single application instance without redundancy, and bounded the volume of AI inference and external API traffic the system could consume
C-04	Platform	The implementation targets a Node.js runtime with TypeScript. Persistence uses a managed PostgreSQL service accessed through a REST client, which does not expose multi-statement transactions across tables; operations spanning several tables therefore require explicit compensating logic. An in-memory data store is required for both job queuing and session storage, making it a hard runtime dependency
C-05	Data availability	The system can only measure what the external tools expose. Several intended metrics are unobtainable in practice such as security alert counts are unavailable where the repository has not enabled the relevant scanning feature, test coverage is reported only where the project's pipeline publishes a coverage report, and certain issue-tracker fields are instance-specific custom fields whose identifiers vary between organisations
C-06	Data history	No historical backfill is possible from the source tools for most metrics. Trend data therefore accumulates only from the first synchronisation onward, meaning the system's analytical value increases over time and is limited immediately after adoption
C-07	Integration access	The system operates strictly read-only against the partner's tools using access tokens, with no ability to install agents, configure webhooks or otherwise modify the partner's infrastructure
C-08	Deployment	The system is deployed to managed cloud platforms which includes a container platform for the server components, a static hosting platform for the client application, and managed cache and database services. At the time of writing it is hosted and operational but has not yet been handed over to SSCL for production use

ID	Category	Constraint and implication
C-09	Confidentiality	Work is conducted under a non-disclosure agreement with SSCL, which governs the handling of their data and the disclosure of information in this report. The obligations arising from it are addressed in Section 3.2

3.2 Regulatory Requirements and Standards

Identify all relevant regulatory requirements, engineering standards, and codes of practice (e.g. data-protection law, WCAG, OWASP, ISO/IEC/IEEE standards, industry rules, partner policies). For each, state why it applies and how the design complies.

3.3 Formulation of Solutions

Formulate multiple genuinely different solutions that could each functionally meet most requirements. Compare and evaluate them against explicit criteria from the requirements and constraints, and draw a valid conclusion that justifies the selected solution.

Chapter 4

System Design and Implementation

4.1 Functional Design and Sub-problem Partitioning

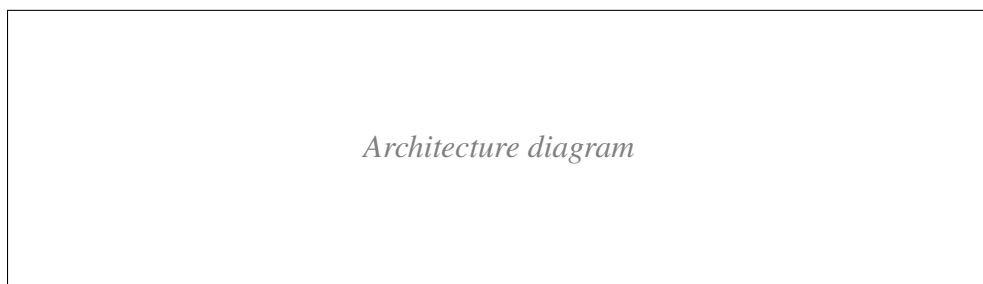


Figure 4.1: System architecture

Partition the problem into sub-problems or modules; explain each one's responsibility and how they interact. Show where the standards and codes of practice from Section 3.2 and any health, safety, or environmental considerations shaped the design.

4.2 Design Refinement and Simulation

Present the detailed design using the views that fit the project (ERD, class, sequence, DFD, API, wireframes, circuit, model). Include any design calculations or estimates made (e.g. expected load or storage), the analyses, prototypes, or simulations used to verify the design against the requirements, and how the design was refined to facilitate implementation.

4.3 Application of Tools

Justify the selection of the important tools and technologies and state their relevant limitations.

4.4 Implementation

Describe how the key modules were implemented and the development process followed (repository, version control, code review, CI/CD).

4.5 Deployment

Describe how the system is deployed and operated: the target environment and infrastructure, configuration and release process, monitoring and backup, and who operates and maintains it. State the computing resources the deployment consumes; their environmental impact is assessed in Section 6.4.

Chapter 5

Investigation and Evaluation

5.1 Experimental Investigation

Present the investigation carried out to substantiate the solution. State the research questions or hypotheses, the method and experimental design, the datasets or measurements used and how they were obtained, the baselines or points of comparison, and the metrics. Analyse and interpret the data, and synthesise the findings into conclusions that are supported by the evidence. Projects with a substantial research and development component should treat this as the core of the chapter; for development-focused projects, report the investigation that validated the main design decisions. This is the experimental work on the built system; the background research is in Chapter 2.

5.2 Performance Evaluation

Evaluate the system against every requirement and objective. Describe the setup, tools, inputs, and acceptance criteria; present the results (tests, measurements, security checks, user or partner acceptance, experiments as appropriate); state which requirements are met and clearly explain each deviation.

5.3 Deviations and Design Revisions

For each deviation, give the technical analysis of its cause and the design revision made to achieve compliance, or explain why it was not revised.

Chapter 6

Project Management, Teamwork, and Impact

6.1 Project Management and Finance

6.1.1 Planning and Risk Management

Present the plan: phases, milestones, task distribution, and schedule, noting where actual progress differed. List the major risks and how they were mitigated.

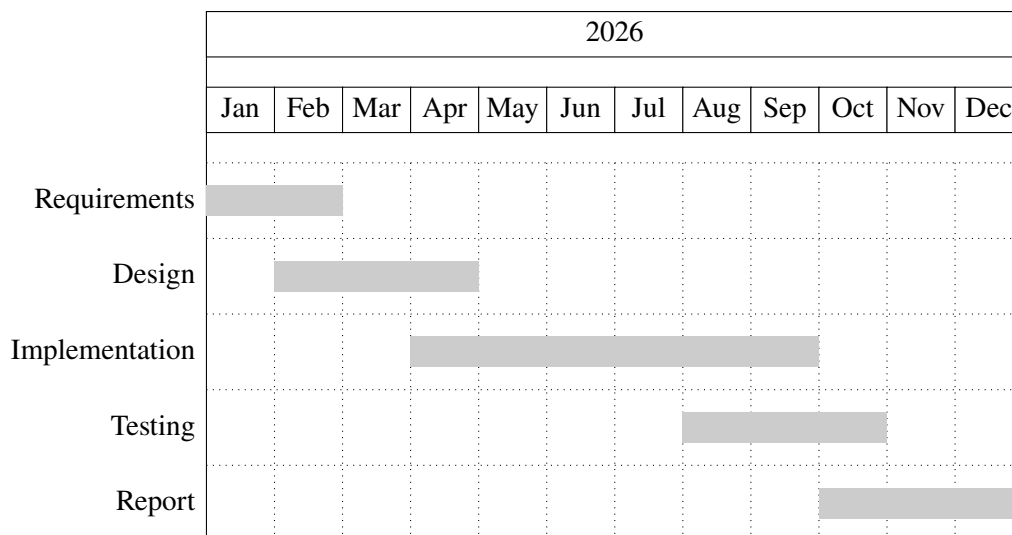


Figure 6.1: Project schedule

6.1.2 Budgeting and Resource Identification

Identify the resources required (people, hardware, software, cloud and API services, data) with an itemised budget.

6.1.3 Economic Analysis and Financial Prospecting

Evaluate the financial prospect of the project with clear interpretation: development and operating cost, the potential users or market established in Section 2.3, and how the solution could generate value or revenue.

6.1.4 Sustainability in Business and Commercialisation

Explain how the solution could remain viable after the course: the commercialisation or deployment path (e.g. with the industry partner) and who would maintain and operate it.

6.2 Individual and Teamwork

6.2.1 Participation and Contribution

For each member, give the role, specific contributions, participation in discussions and decision-making, and whether responsibilities were completed on time.

6.2.2 Collaboration and Conflict Resolution

Describe how the team collaborated and kept a constructive atmosphere, and how any conflicts were identified and resolved.

6.2.3 Leadership and Direction

Explain how direction toward the goals was provided, how members' views were listened to, how the team planned for improvement after setbacks, and how members were motivated.

6.2.4 Multidisciplinary Engagement

Explain the multidisciplinary nature of the project and the team's participation in activities outside its own discipline: work with the industry partner, domain experts, end users, or other departments.

6.3 Impact on Society

Consider all impacts of the solution on society, positive and negative, covering health, safety, cultural, and legal issues explicitly. For each, state who is affected and how the design responds. Be specific to this project.

6.4 Environmental Impact and Life Cycle Sustainability

Consider the environmental and sustainability impacts over the entire life cycle: development, deployment and operation, and retirement of the system and its data. Identify the negative impacts and propose mitigation for each.

6.5 Ethics

6.5.1 Equity and Inclusivity

Show how equity and inclusivity were considered: accessibility, fairness across user groups, and who might be excluded and how that was addressed.

6.5.2 Accountability and Personal Responsibility

Show accountability and personal responsibility: who is responsible for the system's behaviour and failures, and how members took responsibility for their own work.

6.5.3 Proper Use of Intellectual Property

Show due acknowledgement of others' work, prior art, intellectual property, and copyrighted material: cite sources, list third-party code, models, and datasets with their licences, and disclose any generative AI use.

6.5.4 Professionalism and Ethical Codes

Show understanding of the engineer's role in protecting the public interest and awareness of the relevant codes of ethics (e.g. ACM, IEEE, IEB). Explain how they guided specific project decisions.

Chapter 7

Conclusion

7.1 Conclusion and Future Work

Summarise achievement against each objective, the limitations, and focused future work. Do not repeat the abstract.

7.2 Continuous Engagement and Self-directed Learning

Demonstrate engagement in staying up to date: the tools, technologies, and domain knowledge learned independently during the project, and the emerging technical literature or documentation identified and used.